

SEUPD@CLEF: Team Basette

Alberto Bottari¹, Lorenzo Croce¹ and Fatemeh Mahvari Habib Abadi¹

¹University of Padua, Italy

Abstract

This report describes the system developed by Team Basette for the CLEF 2025 LongEval Lab, Task 1 - Web Retrieval. The system retrieves documents using traditional IR techniques enhanced with query expansion; its main focus is performance, in fact both indexer and searcher runs in multithreading for very fast execution speed.

Keywords

Information Retrieval, CLEF, LongEval, Search Engine, Lucene, Multithreading

1. Introduction

Information Retrieval (IR) systems play a fundamental role in enabling access to relevant data across massive, often heterogeneous corpora. As the volume of digital content continues to grow, the effectiveness and adaptability of IR systems are becoming increasingly vital. From search engines and recommendation platforms to digital libraries and research databases, IR technologies underpin a wide array of applications. However, these systems are often sensitive to temporal drift in data and variation in document structures and query formulations, which motivates the need for adaptable, robust retrieval architectures.

In the context of the LongEval Lab at CLEF, which focuses on evaluating the long-term robustness of retrieval systems, we have developed a configurable IR system built in Java and powered by Apache Lucene[1]. Our system supports a wide range of preprocessing, parsing, analysis, and indexing strategies through an extensible configuration framework, enabling us to experiment with different setups.

This paper presents the architecture, implementation, and features of our IR system.

The remainder of this paper is organized as follows: Section 2 describes the methodology and system architecture; Section 3 outlines the experimental setup; Section 4 presents our results and insights; and Section 5 concludes the report with a summary and suggestions for future improvements.

“Search Engines”, course at the master degree in “Computer Engineering”, Department of Information Engineering, and at the master degree in “Data Science”, Department of Mathematics “Tullio Levi-Civita”, University of Padua, Italy. Academic Year 2024/2025

✉ alberto.bottari@studenti.unipd.it (A. Bottari); lorenzo.croce.1@studenti.unipd.it (L. Croce); fatemeh.mahvarihabibabadi@studenti.unipd.it (F. Mahvari Habib Abadi)



© 2025 Copyright for this paper by its authors.

Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

2. Methodology

At the core of the system is the `InformationRetrievalSystem` class, which glues the components of the system together, from preprocessing and parsing to indexing and searching, orchestrating the entire retrieval pipeline. The system is launched via the `Main` class, which parses command-line arguments and initializes configuration paths.

2.1. System Configurability

A central design goal of our submission is that every behaviour-critical decision can be altered without touching the Java sources. To achieve this we encapsulate each tunable subsystem behind a small, serializable **configuration object** that lives in the `de.ir_lab.longeval24.config` package hierarchy (Figure 1).

All the configuration objects implement tiny factory interfaces (such as `IAnalyzerConfig`). These interfaces are annotated so that Jackson [2] can recreate the correct Java class simply from a type field in a JSON file.

At run time the main driver reads the desired configuration file (e.g. `searcher.json`) from the folder provided on the command line and hands it to a shared `ObjectMapper`. The mapper injects runtime services (command-line `Parameters`, an `IndexReader`, etc.) through `@JacksonInject`, then calls the factory's `toRuntime()` method to obtain a fully initialised components. Because of the deserialization of config, changing a setting only requires editing JSON and rerunning the program—no recompilation or re-packaging is necessary.

```
.
├── de.ir_lab.longeval24
│   ├── analyzer
│   │   ├── config
│   │   │   └── analyzer
│   │   ├── filters
│   │   │   ├── agnostic
│   │   │   ├── en
│   │   │   └── fr
│   │   ├── indexer
│   │   ├── parser
│   │   │   ├── document
│   │   │   └── query
│   │   ├── preprocessor
│   │   ├── searcher
│   │   ├── stemmer
│   │   └── tokenizer
│   ├── filters
│   ├── indexer
│   ├── parser
│   │   ├── document
│   │   └── query
│   ├── preprocessor
│   ├── searcher
│   └── utilities
```

Figure 1: Directory structure of the project

2.2. Parallel Computing

An essential way to improve the performance of a search engine is not only by optimizing how queries are executed but also by addressing the time required to index documents[3]. The efficiency of both the indexing process and the execution of user queries is critical, particularly as the volume of documents

grows. In our search engine project, we realized that running numerous experiments with significant hardware constraints required careful attention to the time and resources consumed in these operations. To mitigate performance bottlenecks, we focused on optimizing both the indexing and searching processes using parallelism. This section details how parallelism was implemented to streamline the indexing and searching processes.

2.2.1. Concurrent Indexing Process

In this project, parallelism in the indexing process is implemented using a producer-consumer model. The producer threads are responsible for parsing documents, while the consumer threads handle the indexing of these parsed documents into a Lucene index. Once a document is parsed into a 'ParsedDocument' by a producer, it is placed into a shared 'BlockingQueue', where it will be consumed. For this parallel indexing system, we use an empirical ratio we found to be good for our machine of producer to consumer threads, where 2/5 of the total threads are designated for document parsing (producing) and 3/5 are assigned to document indexing (consuming). Unlike the approach used in the searching process, it became necessary to instantiate multiple producer threads. This decision was based on observations that, for certain analyzer configurations where the size of the index was not significantly larger than the size of the collection, the consumer threads were faster than the producer threads. As a result, performance was wasted during the process due to idle consumer threads waiting for documents to be produced. After all documents are processed, the 'POISON' marker is used to signal the end of the input, ensuring that the consumer threads can terminate gracefully once all documents have been indexed.

2.2.2. Concurrent Searching Process

For the search functionality, parallelism is also employed, again using a producer-consumer pattern. The 'Searcher' class distributes queries across several threads that are concurrently processed. Initially, the 'QueryProducerTask' runs in a separate thread, reading queries from the query parser and placing them into a shared 'BlockingQueue'. Subsequently, multiple 'QuerySearchTask' threads are launched to consume these queries from the queue and execute searches using the Lucene 'IndexSearcher'. Each searcher thread retrieves a query, processes it, and fetches the top-ranked documents from the index. The results are then written to a run file. In the end the 'POISON' marker is used to signal the end of the task, prompting each searcher thread to terminate gracefully.

In Section 4 we present the improvements we achieved by modifying our programs so that it runs on multiple threads.

2.3. Document Parsing

The document parsing component is responsible for transforming raw input files into structured representations suitable for indexing. This process is managed via the `IDocumentParser` interface. It ensures orderly switching between sources and handles resource cleanup. Each document parser implementation is responsible for recognizing and parsing specific document formats.

Supported document sources include:

- `DirectoryDocumentParser`: Recursively scans a directory and processes each file as an individual document.
- `JsonFileDocumentParser`: Parses a single JSON file containing an array of documents.
- `JsonlFileDocumentParser`: Parses documents stored in the JSON Lines format, where each line is a JSON object.
- `JsonGZFileDocumentParser`: Same as `JsonFileDocumentParser` but supports compressed .gz files.
- `JsonlGZFileDocumentParser`: Variant of `JsonlFileDocumentParser` that handles gzip-compressed files.

The `ParsedDocument` class represents a parsed unit with a unique document ID and text content. It includes a `toLuceneDocument()` method that converts it into a Lucene Document.

2.4. Query Parsing

Query parsing is the stage where raw query inputs are read, interpreted, and transformed into structured query objects suitable for execution by the search engine. This functionality is abstracted by the `IQueryParser` interface and implemented in one or more concrete classes. Each query is converted into a Lucene Query object and wrapped with metadata for tracking and scoring. This design allows the parser to be used in iterator-style loops and ensures that both the structured query and its metadata (e.g., query ID and original text) are available to downstream components. The primary implementation of `IQueryParser` in the system is `TxtQueryParser`. It reads a plain text file where each line contains one query, usually in the format:

```
<query-id>\t<query-text>
```

The `RawQuery` class encapsulates metadata associated with a query. It typically includes:

- `qid`: The unique identifier of the query.
- `text`: The raw textual form of the query as provided in the input file.

This metadata is preserved through the search pipeline using the `QueryWrapper` class.

Each line is split into an ID and a query string. The query text is then tokenized and processed using the configured analyzer, and a corresponding Lucene Query object is generated.

The parsed query text may be used to construct various types of Lucene queries, including:

- Term queries
- Phrase queries
- Boolean queries
- Proximity queries

using the configurations, the system can optionally apply query-time filters such as synonym expansion or fuzzy matching if enabled in the analyzer. The final Lucene Query object is passed to the searcher along with the original metadata via the `QueryWrapper`.

2.5. Text Preprocessing

The system provides a mechanism to apply a chain of transformations to input queries or documents before tokenization. This is achieved using the `IPreProcessor` interface, which defines a single `process(String query)` method.

Implementations of `IPreProcessor`

- `UnicodeNormalizerPreProcessor`: Applies Unicode normalization to convert text into a canonical form (NFC).
- `RegexPreProcessor`: Applies regular expression substitutions using a predefined pattern and replacement string.

2.6. Text Analysis

The text analysis phase transforms raw textual input into a stream of tokens that can be indexed and later retrieved. The analysis is performed through the `GenericAnalyzer` class, the main analyzer component, which is dynamically assembled using configuration files specified via implementations of the `IAnalyzerConfig` interface. `GenericAnalyzer` follows a pipeline composed of:

- **Tokenizer:** Responsible for splitting the input text into raw tokens.
- **Token Filters:** Sequentially modify, normalize, or enrich the token stream.
- **Stemmer:** Even if in Lucene it's implemented as a token filter the importance of this component is crucial so we decided to make it explicitly configurable. This component is in charge of removing suffixes and word endings.

As mentioned before, all these components are configured externally and combined at runtime, enabling high flexibility in how different corpora or languages are handled.

2.6.1. Tokenizer Configurations

Three tokenizer configurations are supported:

- `StandardTokenizerConfig`: Uses Lucene's standard tokenizer, which relies on Unicode Text Segmentation rules to handle punctuation, digits, and word boundaries.
- `LetterTokenizerConfig`: Emits tokens that consist only of alphabetic characters.
- Non-letter characters act as delimiters.
- `WhitespaceTokenizerConfig`: Splits input strictly on whitespace characters.
- It does not process punctuation or symbols.

2.6.2. Token Filters

We have utilized Token Filters to expand and modify the token streams of both queries and documents. These filters are categorized into agnostic and language-specific types, and they are applied after tokenization. By doing so, they provide a measure of control over token processing and normalization.

Agnostic Token Filters These filters are language-independent and perform general text normalization or enhancement operations:

- `LowerCaseFilterConfig`: Converts all tokens to lowercase, ensuring case-insensitive search.
- `ASCIIFoldingFilterConfig`: Transforms accented and special characters into their closest ASCII equivalents.
- `TrimFilterConfig`: Removes any leading or trailing whitespace around each token.
- `LengthFilterConfig`: Discards tokens that are too short or too long based on configured thresholds.
- `NgramFilterConfig`: Generates character-level n-grams to enable fuzzy or partial term matching.
- `RegexFilterConfig`: Uses regular expressions to remove or modify tokens based on pattern matching.
- `SpellCheckerFilterConfig`: Applies Lucene's spell-checking module to suggest corrections for isolated tokens using a predefined dictionary.
- `WordnetTokenFilterConfig`: Expands tokens using WordNet-based synonym lookup, informed by part-of-speech tagging.

Language-Specific Token Filters These filters handle language-specific linguistic features and are designed for either English or French:

- `FrenchStopFilterConfig`: Removes common French stopwords such as “le”, “de”, and “et”.
- `FrenchElisionFilterConfig`: Removes French elisions in contractions (e.g., “l’homme” becomes “homme”).
- `EnglishStopFilterConfig`: Removes frequent English stopwords such as “the”, “and”, and “of”.
- `EnglishPossessiveFilterConfig`: Removes possessive suffixes (e.g., “company’s” becomes “company”).

These filters are especially useful when working with multilingual corpora, allowing token pipelines to be adapted accordingly.

2.6.3. Stemming and Synonym Expansion

Stemming is available through built-in Lucene stemmers configured within the analyzer. The system supports both standard and minimal stemming strategies for English and French. Stemming reduces words to their root forms (e.g., “running” to “run”), enhancing generalization during matching.

Synonym expansion is implemented in two complementary ways. The first method is **static synonym expansion**, which utilizes the `SynonymAnalyzer`. This approach loads synonym lists from files and applies them during query processing. The second method is **dynamic WordNet-based expansion**, which employs the `SynonymPOSAnalyzer`. This analyzer applies part-of-speech tagging using OpenNLP, and then it looks up appropriate synonyms from WordNet. This approach helps to avoid polysemy errors and enables semantic enrichment of queries. Both methods have been written as code but have not yet been tested.

Synonyms are added with lower weight than original tokens to preserve ranking integrity while improving recall. These expansions are applied only at query time to avoid excessive index bloat.

2.6.4. Ngrams

`ShingleFilter` class in Lucene is a `TokenFilter` that generates n-grams from a stream of tokens. It is commonly used to enhance the representation of text by creating overlapping token sequences of a specified size, which can capture contextual information and improve recall[?]. In our information retrieval system, we are using the `ShingleFilter` during both the indexing and query phases. During indexing, it helps create a richer index by storing token n-grams, while during query expansion, it enriches the query by adding n-gram variations of the original query terms. This approach, often referred to as n-gram expansion, helps improve the effectiveness of search by expanding the terms that are matched, especially for queries that might be too specific or lack exact term matches.

2.7. Indexing

The indexing stage is responsible for transforming processed documents into an inverted index using Lucene. This index enables efficient retrieval of relevant documents in response to user queries. The indexing logic is encapsulated in the `Indexer` class, which is instantiated at runtime based on external configuration.

2.7.1. Indexer

The `Indexer` class serves as the core component responsible for the indexing workflow. Its main function is to iterate through all parsed documents, apply the configured analyzer to the document content, and write the resulting tokens into a Lucene index.

The indexer operates on a designated input directory and outputs to a target index directory. It is responsible for several tasks related to indexing, including initializing a `Lucene IndexWriter` with the appropriate configuration, processing each document using the provided analyzer, and storing fields such as the document ID and body content. Additionally, it records term positions and frequencies to support proximity queries and scoring models like BM25[4]. The indexer also logs important statistics, including the number of files processed and the time taken for the indexing process.

The indexer supports token storage options that include term frequency and position tracking.

2.7.2. Index Field Configuration

The indexer writes each document's content into a dedicated Lucene field, typically named `BODY_FIELD`. This field is configured to store term frequency and position information, which is used for ranked retrieval and phrase queries. Additionally, it can optionally store the original body text, which may be useful in downstream components such as rerankers.

2.7.3. Indexer Lifecycle

The lifecycle of the indexer follows a standard pattern. First, the indexer is instantiated via configuration. Then, the Lucene index is opened for writing. For each document, the analyzer is applied, a Lucene `Document` object is created, and the document is added to the index. Finally, changes are committed, and the index is closed.

This design abstracts away the details of Lucene's indexing API, making it easy to plug in different analyzers or indexing strategies without changing core logic.

2.8. Searching

The searching stage is responsible for executing user queries over the previously created Lucene index and retrieving the most relevant documents. This process is implemented through the `Searcher` class, which orchestrates multi-threaded query execution and writes ranked results in TREC run format.

The `Searcher` class serves as the main retrieval engine and is instantiated with several key components. These include a `Lucene IndexReader` for accessing the index, a similarity model (such as BM25) to score documents, and a query parser that generates Lucene queries from raw query text, as discussed in Section 2.4. Additionally, the `Searcher` class is configured with settings that define the output path, retrieval depth, and runtime parameters.

The searcher retrieves a fixed number of top-ranked documents per query and writes the results to an output file.

2.8.1. Reading the Query

The `QueryWrapper` is a simple utility class that pairs a parsed query (with metadata like query ID) and its corresponding Lucene `Query` object. It allows the system to retain contextual information throughout the retrieval pipeline.

To accelerate retrieval, the search process is parallelized using multiple threads and follows a producer-consumer architecture involving two key classes. The `QueryProducerTask` runs in a dedicated thread and iterates over all queries using the configured parser. It converts the queries into Lucene queries, wraps them in `QueryWrapper` objects, and places them into a shared queue. The `QuerySearchTask` runs in multiple threads, with each thread consuming `QueryWrapper` objects from the queue, executing the search using Lucene's `IndexSearcher`, formatting the results, and writing them to the output. By default, the number of search threads is set to match the number of available CPU cores minus one. A poison-pill sentinel object is used to signal thread termination once all queries have been processed.

2.8.2. Similarity Model

The system uses Lucene's BM25 implementation as the default similarity model. Parameters such as k_1 and b are configurable. The searcher is designed to support swapping the similarity model via configuration, enabling experimentation with other ranking algorithms if needed.

3. Experimental Setup

The experimental conditions under which our information retrieval system was developed, deployed, and evaluated are as follows:

- **Used Collections:** All experiments were conducted using the LongEval CLEF 2025 Lab dataset [5], which included documents in various formats (such as `.json`, `.trec`, etc.), queries in plain text format, and the corresponding relevance judgement files.
- **Evaluation Measures:** System performance was assessed using standard information retrieval metrics included in the `trec_eval` tool.
- **Source Code Repository:** The full source code is available at: <https://bitbucket.org/upd-dei-stud-prj/seupd2425-basette>
- **Hardware Used for Experiments:** All experiments were executed on a consumer-grade desktop PC with the following specifications:
 - CPU: Intel Core i3, 10th Generation (4 threads)
 - RAM: 16 GB DDR4
 - GPU: Not utilized
 - Storage: SSD (unspecified size)
 - Operating System: Ubuntu 24.04.2 LTS (within Windows 11 through WSL)
- **Software Environment:** The system was run with the following environment:
 - Java 17 (OpenJDK)
 - Maven 3.8.6
 - Python 3.11 (for document formatting utilities)
 - Lucene 9.0.0

A Docker container was used to hold an image of a LaTeX library for compiling this report.

- **Run Procedure:** For detailed information about the steps required to run the project please refer to the `readme.md` file in the BitBucket repository.

Table 1
Indexing Performance on Document Dataset

Indexing Approach	Document Dataset Size (GB)	Index Size (GB)	Average Time (s)
Single Thread	12	12	1800
Multithread	12	12	176

4. Results and Discussion

In order to compare the results of different configuration of components and find the best-performing one, we have selected a array of different values for each configuration and through the experiments that will be presented in the following sections, aimed to fine-tune our system to the best of its capabilities.

For the purposes of testing our system, the following parts of the CLEF’s collection were used:

- 2022-07_fr
- 2022-11_fr
- 2022-02_fr

4.1. Indexing

The performance results of our indexing process, as shown in Table 1, highlight the significant improvements achieved through multithreading. When using a single-threaded approach, indexing a 12 GB document dataset took approximately 1800 seconds, whereas with multithreading, the same task was completed in just 176 seconds, demonstrating a substantial reduction in indexing time.

However, the effectiveness of multithreading varied depending on the type of analyzer used during the indexing process. For example, when using n-grams as an analyzer, the size of the index increased significantly, by almost a factor of 3. This expansion in index size resulted in storage becoming a bottleneck, which reduced the impact of multithreading. In such cases, the benefits of parallelism were less noticeable because the increased storage requirements limited the overall performance gains.

On the other hand, during the early stages of testing, where we focused on optimizing the BM25 algorithm by applying simple filters such as stopword removal, trimming, and French elision—filters that did not significantly increase the index size—we saw a marked improvement with multithreading. In this scenario, the reduced indexing time was more pronounced because the index size remained manageable, allowing the parallelization of tasks to effectively speed up the process without being blocked by storage constraints.

Thus, the impact of multithreading on indexing performance is heavily influenced by the size of the index and the types of analyzers and filters applied. When the index size remains small and manageable, parallelism offers substantial performance gains, but as the index grows, particularly with resource-intensive analyzers like n-grams, storage becomes the limiting factor.

4.2. Analyzer

We first focus on the analyzer component of our search engine, which plays a crucial role in the pre-processing of documents. In this context, we compared two different analyzers: the default French analyzer provided by Lucene and the custom analyzer that we implemented with our own set of filters. The performance of both analyzers was evaluated across several test collections.

Our results, summarized in the Precision-Recall Curve metrics, in Figure4, Figure2 and Figure3, show that our custom analyzer outperforms the default Lucene analyzer across all tested datasets. Specifically, our analyzer demonstrates a slight improvement in performance, with a higher AUC score in all experiments conducted.

Although the performance gain among the use of trim and regex is very small, expected given the relatively “clean” nature of the documents, it reinforces the hypothesis that further cleaning of

the documents can lead to small, but measurable improvements in retrieval quality. This finding is not surprising, as we would anticipate that the application of additional cleaning techniques (such as trimming and regex filtering) would not degrade performance, but rather enhance it marginally.

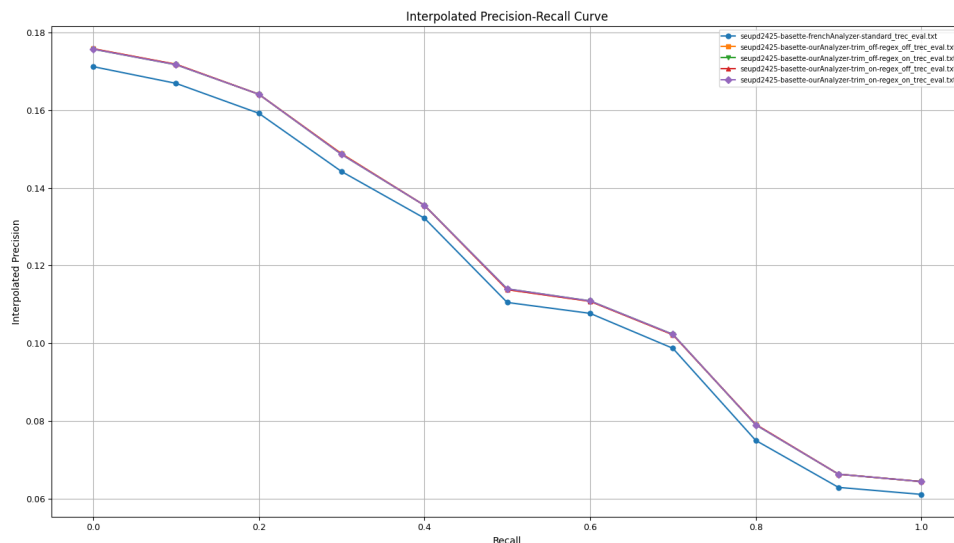


Figure 2: Precision-Recall curves comparing the default Lucene French analyzer with our custom analyzer using 2022-07_fr collection

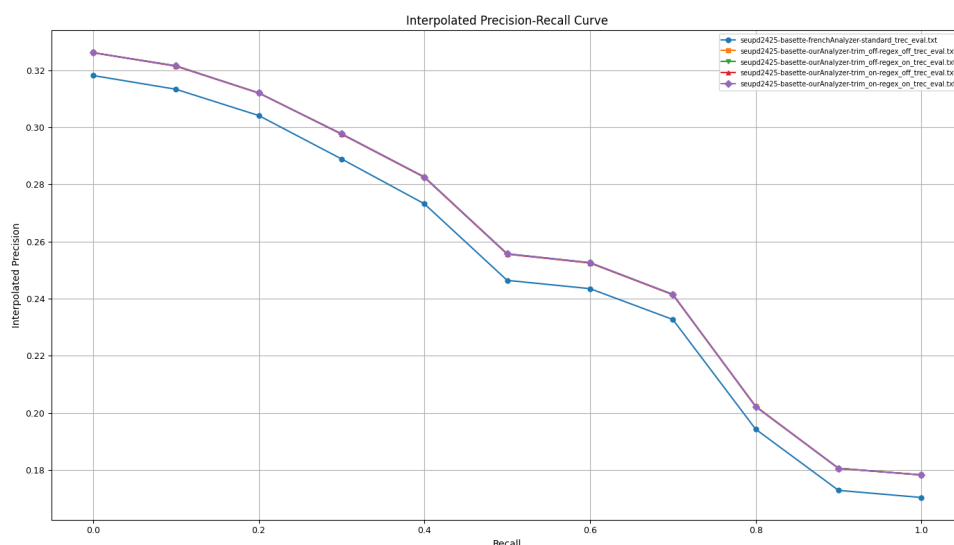


Figure 3: Precision-Recall curves comparing the default Lucene French analyzer with our custom analyzer using 2022-11_for collection

These results affirm the logical expectations: improving the cleanliness of the documents through additional filtering techniques leads to slight, yet tangible improvements in retrieval performance. Therefore, it can be concluded that our custom analyzer, with its additional filters, provides a slight but consistent benefit over the default Lucene analyzer.

4.3. BM25 Similarity and Parameter Tuning

In this section, we examine the impact of the BM25 similarity function, focusing on the influence of its two key parameters: k_1 and b_1 . The BM25 ranking function is commonly used in document retrieval, with its parameters governing the balance between term frequency and document length

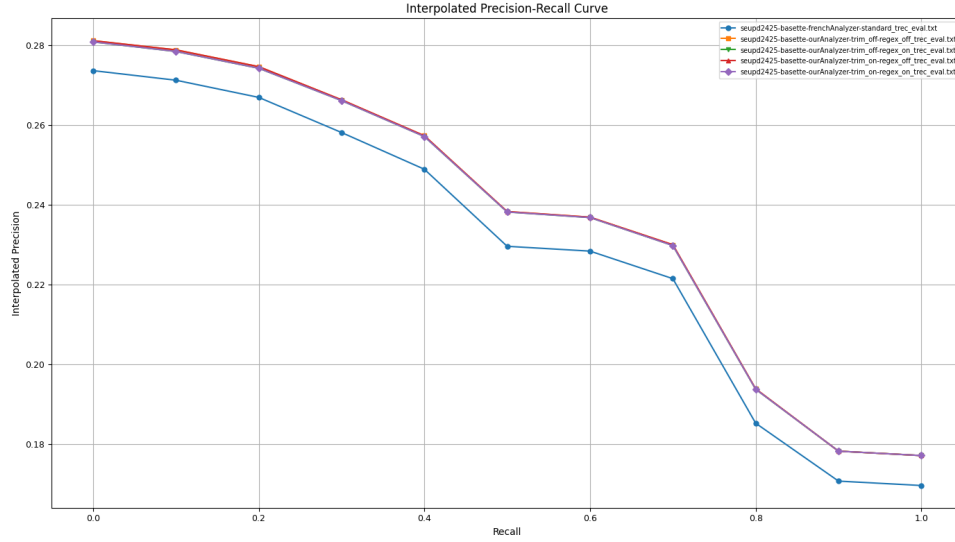


Figure 4: Precision-Recall curves comparing the default Lucene French analyzer with our custom analyzer using 2023-02_fr collection

normalization. Specifically, k_1 controls the scaling of term frequency (TF), while b_1 adjusts the degree of length normalization.

Our experiments were conducted across three different collections, with the goal of identifying the values of k_1 and b_1 that maximize retrieval performance, as measured by Mean Average Precision (MAP) and Normalized Discounted Cumulative Gain (NDCG). The results, shown in Figure5 and Figure6, indicate that the optimal values for the parameters are $k_1 = 1.4$ and $b_1 = 0.95$, which are relatively central within the heatmaps and lead to improved MAP and NDCG scores across all three collections.

The value of $k_1 = 1.4$ aligns with theoretical expectations, where increasing k_1 (up to a certain point) enhances the term frequency component, making frequent terms more influential in the ranking. Similarly, the value of $b_1 = 0.95$ is relatively high compared to the standard theoretical recommendation, indicating that longer documents are not penalized as heavily for their length. This adjustment allows longer documents to compete better with shorter ones.

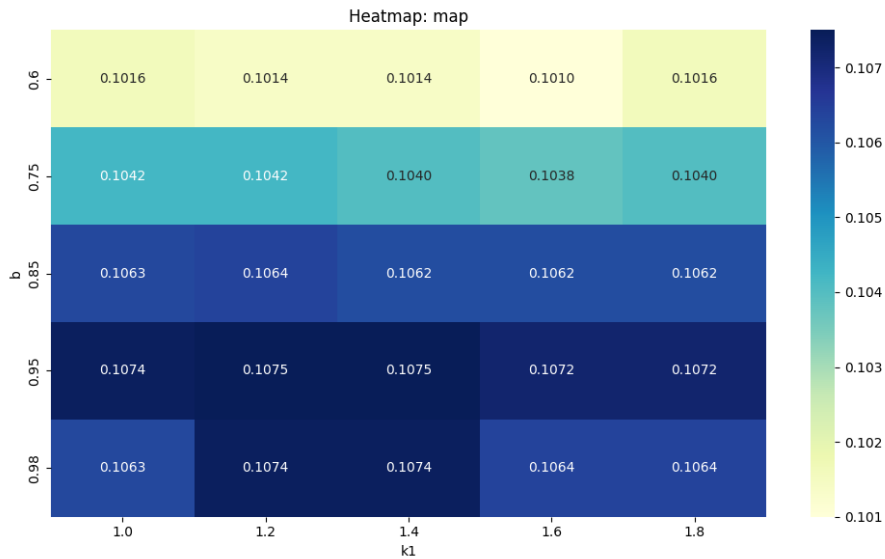


Figure 5: Heatmap for MAP across different values of k_1 and b_1 for the mean of the three collections.

However, it's important to note that certain issues with the dataset and its associated relevance

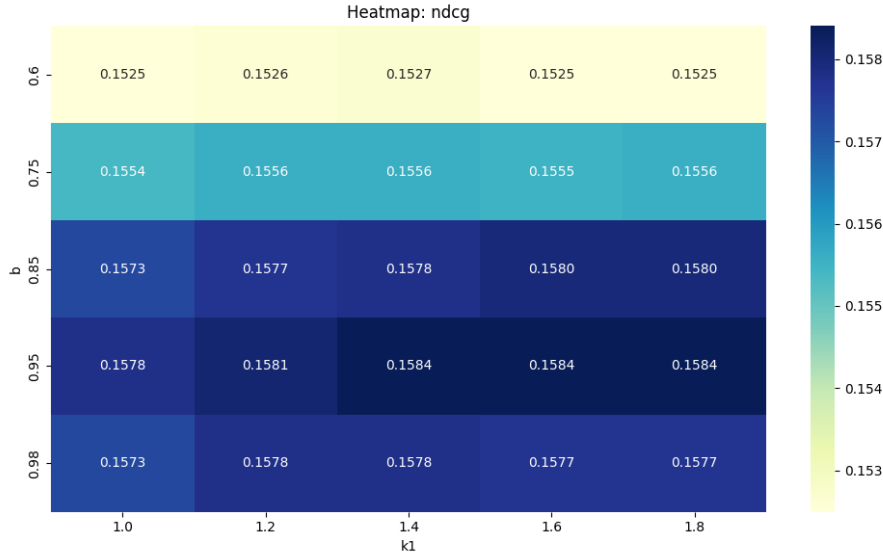


Figure 6: Heatmap for NDCG across different values of k_1 and b_1 for the mean of the three collections.

judgments (qrels) may have influenced these results. In particular, some queries were matched with documents that were not truly relevant or related to the query. This misalignment between query-document pairs might have been caused by a miscalibration in the query-to-document association process. Consequently, while the reported values of $k_1 = 1.4$ and $b_1 = 0.95$ seems optimal, further refinement of the dataset of the qrels could lead to different findings.

4.4. N-grams

N-grams are sequences of n items, typically words or characters, extracted from text, widely used in natural language processing (NLP) to analyze linguistic patterns and local word relationships. They are effective due to their simplicity, computational efficiency, and ability to support various tasks like text classification, language modeling, and spell correction without requiring deep linguistic understanding. By capturing short-term context, n-grams allow systems to make useful predictions about language and improve search capabilities.

However, n-grams come with trade-offs. As n increases, the number of possible combinations grows exponentially, leading to issues like data sparsity and increased memory usage. Smaller n-grams may miss important contextual meaning, while larger ones require significantly more data and computational resources to be effective. Despite these limitations, their practicality keeps them relevant, especially when speed and interpretability are prioritized.

In our specific case, the main trade-off involved the impact of n-grams on indexing performance. Their use significantly slowed down the indexing process due to the increased volume of data generated and processed. Nevertheless, this drawback was offset by improving the AUC of the search engine.

4.5. More infos

To view the details of our TREC Eval runs, you can visit the following link: <https://bitbucket.org/upd-dei-stud-prj/seupd2425-basette/src/master/eval/>. We have uploaded all of our TREC Eval runs across various parameters, providing a comprehensive overview of the results.

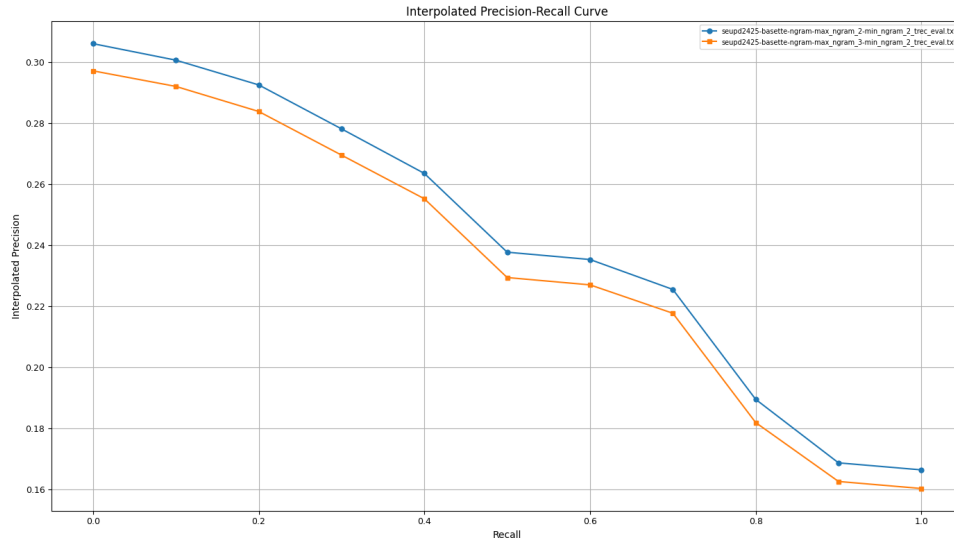


Figure 7: Precision Recall curve of the N-gram trec evaluation with different parameters

5. Conclusions and Future Work

In this project, we explored various aspects of information retrieval within the CLEF LongEval framework, focusing on configurable and performance-oriented approaches. Our main achievements include building a modular and multithreaded system, fast even under hardware limitations.

Unfortunately, during the course of the project, our group faced a major setback: out of the originally planned team, only half of the members remained actively involved. This reduction in personnel significantly limited the time and resources available, impacting our ability to run more experiments and implement additional planned features. As a result, the classes we developed to include query expansion in our system—by using n-grams and synonyms—unfortunately had to be abandoned during the final experiments and were left for future work. Despite this, we prioritized the flexibility and efficiency of our system, laying a solid foundation for future extensions.

It is also important to note that the dataset used for training and evaluating our system initially contained several issues, which affected both the quality and consistency of our experimental results. These problems were only resolved less than a day before the final submission deadline, leaving minimal time for thorough re-evaluation or retraining. As a result, some performance assessments may not fully reflect the system’s true performance. We consider this an important area for follow-up work.

Looking ahead, there are several areas for future development. First, we plan to evaluate alternative similarity measures beyond BM25—such as classical TF-IDF or Dirichlet smoothing—to investigate their impact on retrieval performance. We also aim to experiment with different stemmers and stopword dictionary sizes to fine-tune the preprocessing pipeline.

To improve relevance ranking, we intend to incorporate reranking methods and Named Entity Recognition (NER). In particular, we aim to adopt a pretrained transformer-based model such as BERT[6] for reranking the top retrieved documents.

The CLEF LongEval focuses on long-term performance and resilience to changes in query patterns over time; Incorporating named entity recognition (NER) could play a big role in adapting to shifts in query behavior. Given the modular design and flexibility of our system, we’re confident that these enhancements are feasible.

6. Group Members Contribution

For educational purposes, the team distributed the workload equally to ensure that each member gained hands-on experience with every component of an IR system, as well as with writing the report. In addition to these shared responsibilities, each member specialized in a specific component of the project:

Bottari Alberto Focused on designing and implementing document and query parsing mechanisms, as well as integrating multithreading in the search and indexing process.

Croce Lorenzo Worked on configuring custom and Lucene utilities, and contributed to the multithreading implementation in search and indexing processes.

Mahvari Habib Abadi Fatemeh Specialized in query and document preprocessing, including the development of spellchecker, shingles, and WordNet token filters; also set up the LaTeX Docker environment.

References

- [1] Apache, 2023, Lucene v9.5.0, URL: https://lucene.apache.org/core/9_5_0/index.html.
- [2] FasterXML, 2025, Jackson (GitHub Repository), URL: <https://github.com/FasterXML/jackson>.
- [3] S. Hariri, L. Kurgan, Parallel Computing and Information Retrieval, Springer, 2004.
- [4] S. Robertson, H. Zaragoza, The probabilistic relevance framework: Bm25 and beyond, Foundations and Trends in Information Retrieval 3 (2009) 333–389. doi:10.1561/15000000019.
- [5] C. L. Organizers, 2024, CLEF LongEval Data, URL: https://lucene.apache.org/core/9_5_0/index.html.
- [6] H. Face, 2025, BERT model documentation – Transformers, URL: https://huggingface.co/docs/transformers/en/model_doc/bert.